Bounded-memory streaming redaction of personal data and credentials with a whole-document equivalence guarantee

Mounish Senisetty[a,*], [MISSING: co-author full name, or delete this line][a], [MISSING: co-author full name, or delete this line][a]

[a] *[MISSING: Department, Institution], [MISSING: City], [MISSING: Country]*

Abstract

Text artefacts that leave an organisation, such as application logs, database exports and packet captures, carry personal data and credentials that must be removed before sharing, and are often too large to hold in memory. Processing them in chunks introduces a failure mode that stays invisible until it leaks: a sensitive value straddling a chunk boundary reaches the detector in two halves, matches nothing, and is written out in the clear. We present SafeStream-Redactor, a local redaction engine that treats this as a correctness problem rather than an engineering detail. Under two stated locality assumptions, bounded span length and bounded context, with constants derived from each detector's own declaration, we prove that its streamed output is identical to a whole-document pass, that its multi-process output is identical to the sequential pass or falls back to it, and that its working memory is independent of input size. Peak memory added by streaming stayed under 8.6 MB from 8 MiB to 2 GiB of input, against 905 MB for a 32 MiB whole-document pass; the Rust core redacted a 256 MB log at 10.5 MB/s on one worker and 41.1 MB/s on four. On held-out test splits the pattern tiers fully redacted more credentials than gitleaks and TruffleHog (0.437 against 0.311 and 0.270) at statistically indistinguishable precision. With the same spaCy model SafeStream exceeded Presidio in character F1 (0.702 against 0.650); against Presidio's larger model the entity-coverage difference was not significant. Free-text recall without a named-entity model is low, and we report

*Corresponding author.
Email address: senisettymounish@gmail.com (Mounish Senisetty)

it.

Keywords: data redaction, secret detection, personally identifiable information, streaming algorithms, bounded memory, data loss prevention

---

1. Introduction

Text artefacts that leave an organisation, such as application logs attached to a support ticket, a database export handed to an analyst, or a packet capture shared with a vendor, routinely carry two classes of sensitive value: personal data (e-mail addresses, telephone and card numbers, national identifiers, names) and credentials (API keys, access tokens, private keys, passwords). Data-loss-prevention systems inspect such content before it crosses a boundary [1], and large-scale studies have shown that credentials leak into shared artefacts at scale [2, 3].

Two families of tools address the problem from opposite sides. General-purpose personal-data engines such as Microsoft Presidio [4] combine pattern recognisers with a statistical named-entity model, but ship no credential recognisers. Secret scanners such as gitleaks [5] and TruffleHog [6] target credentials in source code, and their accuracy varies widely across secret types [7]. Neither family states what happens when the input is larger than memory.

Size is the practical obstacle. A multi-gigabyte log cannot be loaded and analysed in one call, so it must be processed in pieces. A naive chunked redactor has a failure mode that is invisible until it leaks: a value that straddles a chunk boundary is presented to the detector in two halves, neither of which matches, and both halves are written out in the clear. Overlapping windows reduce the risk, but without an argument about how much overlap is enough, and about how context near a boundary affects detection, the output of a chunked run can differ from the output of a whole-document run.

This paper describes SafeStream-Redactor (hereafter SafeStream), an open-source engine that treats chunked execution as a correctness problem. Its contributions are:

1. A streaming redaction engine whose output is provably identical to a whole-document pass under two stated assumptions, bounded span length (A1) and bounded context (A2), with the assumption constants

derived mechanically from each detector's declared requirements and pinned by regression tests (Section 4.5).

2. A multi-process mode whose output equals the sequential output byte for byte, enforced by a seam cross-check that falls back to sequential processing on any disagreement, and a working-memory bound independent of input size (Section 4.5).
3. A detection cascade covering personal data and credentials in one pass, with a resolver that never leaves part of a value exposed when detections overlap (Section 4).
4. An evaluation on held-out test splits against Presidio and two purpose-built secret scanners, with bootstrap confidence intervals, a component ablation, a contamination analysis for a benchmark used during development, and a memory scaling curve (Sections 5–6).

[MISSING: If the FTNCT'09 conference paper on SafeStream-Redactor was published, cite it here and state the new content of this article: held-out test evaluation, secret-scanner baselines, ablation, contamination analysis, peak-RSS memory curve, benign-shape suppression in the entropy tier, native core parity results.]

Two scope limits are worth stating at the outset. First, the guarantee concerns equivalence rather than accuracy: streaming does not improve detection, it makes the streamed result the same as the whole-document result. Detection quality is reported separately and without favourable selection. Second, our evaluation has a known weakness that we quantify rather than omit. One of the three corpora, RedactionBench, was inspected during rule development, so its test half is not held out; Section 6.3 reports pre-exposure and post-exposure results side by side and bounds the contamination. On free text without a named-entity model, SafeStream's recall is low, and against Presidio's larger model its entity coverage is not significantly better. We report both.

2. Related work

Personal-data detection and de-identification. Rule-based de-identification of clinical text dates back to lookup tables and regular expressions applied to nursing notes [8]; Philter refined this approach and reported high recall on a large annotated clinical corpus [9]. Shared tasks such as the 2014 i2b2/UTHealth de-identification track established annotated corpora

and evaluation practice [10], and recurrent neural networks reached strong results on them [11]. Outside medicine, Presidio [4] combines pattern recognisers with spaCy or transformer models and ships an evaluation toolkit with a synthetic dataset [12]. Lison et al. argue that sequence labelling alone does not amount to anonymisation and call for explicit disclosure-risk measures [13]; the Text Anonymization Benchmark operationalises this for court judgments [14]. Named-entity recognition underpins the free-text part of these systems, from the CoNLL-2003 shared task [15] to transformer encoders [16, 17] and generalist span models [18]. These systems analyse a document, or a fixed-size piece of one; none of the works above states a relationship between chunked and whole-document output. Pseudonymisation with a keyed hash such as HMAC [19] preserves referential integrity but reveals equality between values, and is therefore not anonymisation in the sense of k-anonymity [20].

Credential and secret detection. Meli et al. measured secret leakage across public GitHub repositories and showed that leaked keys appear daily [2]. Detection approaches range from vendor-specific regular expressions [21] to machine-learned filters that reduce false positives [22] and neural detectors for free-form passwords [23]. SecretBench provides a labelled corpus of candidate secrets [24], and a comparison of nine detection tools found false positives from generic patterns and entropy heuristics and false negatives from missing rules [7]. Developers' own strategies for handling leaked secrets are studied by Krause et al. [3]. Shannon entropy [25] is the usual signal for undocumented high-entropy secrets. The rule set of Nosey Parker [26] supplies the labelled examples we evaluate on.

Streaming and matching. Processing unbounded input in sub-linear memory is the setting of the data-stream model [27]. Linear-time regular-expression matching goes back to Thompson's construction [28]. Our contribution is not a new matching algorithm; it is an overlap and seam protocol that makes a windowed detection pipeline provably equal to its whole-document form, together with an engineering discipline that keeps the required constants true as detectors change. Among the redaction, de-identification and secret-scanning systems cited above, we found none that states such an equivalence or bounds the overlap it needs; the closest practice is ad hoc overlapping of windows, whose correctness depends on how long the longest entity and its context are, which is exactly what our assumptions make explicit.

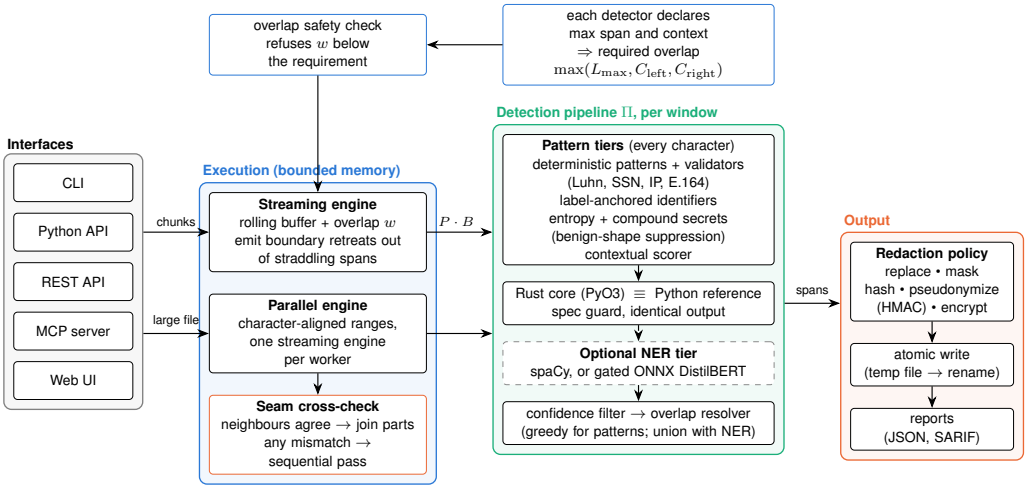


Figure 1: System architecture. Every interface feeds text to the streaming engine, or, for a large file, to the parallel engine, whose workers each run a streaming engine and whose seams are cross-checked. Each window passes through the detection pipeline Π: the pattern tiers run on every character, on the Rust core or the identical Python reference; a named-entity tier is optional (dashed). The resolver produces disjoint spans, which the redaction policy rewrites and an atomic write commits. The overlap w is checked against the maximum span and context that the active detectors declare.

3. Problem and threat model

The input is a text T, possibly larger than memory, arriving as an arbitrary sequence of chunks. The output must be T with every sensitive span replaced. The adversary is a later reader of the redacted output; a *leak* is any character of a sensitive span that remains in the clear. A partially redacted value is treated as a leak, because a card number or key with a few visible characters can often be reconstructed or narrowed. Detection errors are outside the streaming guarantee; the guarantee targets the additional errors that chunking itself would introduce. The system runs locally and makes no network calls, so the unredacted text never leaves the host.

4. Method

Figure 1 shows the architecture. A detector maps a string to a list of detections (i, j, τ, c): a half-open character span, an entity type and a confidence. The *reference pipeline* Π runs every configured detector on a text,

adjusts confidences with a bounded contextual scorer, drops detections below a threshold (0.5 by default) and resolves overlaps into a disjoint set. Redaction is a deterministic rewrite of T given $\Pi(T)$.

4.1. Detection tiers

Deterministic patterns with validators. Credential formats with a fixed shape (cloud access keys, provider tokens, JSON Web Tokens, PEM private-key blocks) and structured personal data (e-mail, telephone, card and social-security numbers, IPv4 and IPv6 addresses) are matched by regular expressions. A match becomes a detection only if it passes a validator: the Luhn checksum for card numbers [29], area, group and serial rules for social-security numbers, octet ranges for IPv4, address parsing for IPv6, and an E.164 shape check for telephone numbers. A generic key-assignment rule matches values that follow keys such as `api_key`, `token` or `password`.

Label-anchored identifiers. Identifiers with no checksum (patient, account or licence numbers) are ambiguous in isolation. This tier promotes a value to a detection only when a cue word that names it (for example `Patient ID:`) appears within 96 characters to its left.

Entropy tier with benign-shape suppression. Candidate runs of 20 to 8192 characters over a key alphabet are scored by length, character-class mix and Shannon entropy [25], with a lower threshold for hexadecimal runs. A compound variant covers runs with embedded symbols. Entropy alone cannot tell a secret from other random-looking strings. During this study we found that every generated UUID, commit hash, SHA-256 digest and base64 image payload in the secrets development split was flagged (Section 6.2). The tier therefore suppresses three shapes: a canonical UUID, a 40- or 64-character hexadecimal value preceded by a digest cue (`commit`, `sha256`, `hash` and similar), and the payload of a `data:` URI. Suppression is vetoed when a secret-like key (`key`, `token`, `secret`, `password` and similar) names the value, so `SAUCE_ACCESS_KEY=`*uuid* remains a secret. The check reads at most 32 characters of left context.

Contextual scorer. A 48-character left window raises or lowers confidences when trigger words are present, and emits a small set of trigger-gated detections of its own.

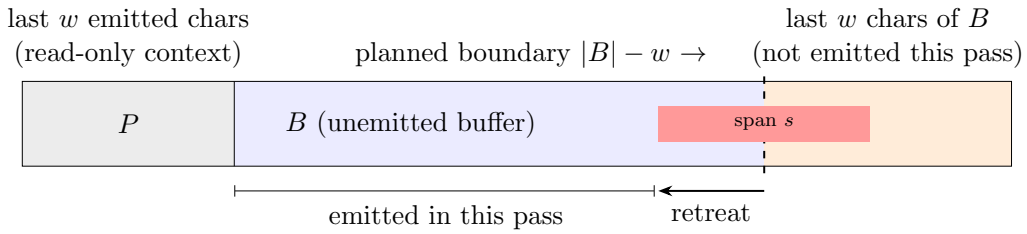


Figure 2: One pass of Algorithm 1. The detector reads $P \cdot B$; only text before the planned boundary may be emitted, and the boundary retreats out of any span that crosses it, so s is carried to the next pass and detected whole.

Optional named-entity tier. For names, organisations and locations, an optional tier runs a spaCy pipeline (`en_core_web_sm` in our experiments). An ONNX DistilBERT tier [17] is also implemented but was not evaluated in this study (Section 7).

4.2. Overlap resolution

Pattern detections that overlap are resolved best-first by confidence, then by length, so a card number beats a telephone number embedded in it. Named-entity detections are handled differently: any cluster of overlapping spans that contains a named-entity span becomes one span from the cluster's leftmost start to its rightmost end, labelled by its best pattern member if one exists. Without this union rule, a PERSON tag on `john.doe` could replace the e-mail detection for `john.doe@example.com` and leave the domain exposed.

4.3. Streaming engine

Algorithm 1 (illustrated in Figure 2) keeps a buffer of unemitted text and the last w characters already emitted. Each pass detects over the concatenation, so detectors see real left context rather than an artificial edge, and emits only the part of the buffer that lies at least w characters before its end. If a detection crosses the planned emit boundary, the boundary retreats to the detection's start and the detection is carried into the next pass. Small chunks are coalesced until at least m characters can be emitted, so the cost of rescanning the overlap is bounded per emitted character regardless of how the input is chunked. The defaults are $w = 8192$ characters and $m = 65{,}536$ characters.

Algorithm 1 Streaming detection with a bounded overlap w (simplified from `engine.stream_windows`).

Require: chunks $c_1, c_2, \ldots$; detector D; overlap w; minimum emit size $m \geq w$

$P \leftarrow \varepsilon$ ▷ already emitted text, kept as read-only left context
$B \leftarrow \varepsilon$ ▷ buffer of unemitted text
for each chunk c **do**
  $B \leftarrow B \cdot c$
  if $|B| < w + m$ **then continue**
  end if
  $S \leftarrow \{(i - |P|, j - |P|, \cdot) : (i, j, \cdot) \in D(P \cdot B),\ i \geq |P|\}$ ▷ drop spans inside P
  $e \leftarrow |B| - w$
  while some $(i, j, \cdot) \in S$ has $i < e < j$ **do** $e \leftarrow i$
  end while ▷ retreat out of straddlers
  if $e \leq 0$ **then continue**
  end if ▷ a span pins the buffer; read more
  emit $B[0{:}e]$ with $\{s \in S : s.j \leq e\}$
  $P \leftarrow$ last w characters of $P \cdot B[0{:}e]$; $B \leftarrow B[e{:}]$
end for
emit B with the spans of $D(P \cdot B)$ that start at or after $|P|$

4.4. Parallel engine

For large files, the parallel engine splits the input into character-aligned byte ranges, one per worker process. Each worker reads w characters of scanned padding and a further w characters of context-only prefix around its interior boundaries (seams). A span that crosses a seam belongs to the left worker, and the right worker begins emitting at that span's end. Neighbouring workers compute the seam position independently, and the coordinator compares their reports. On any disagreement it discards the parallel result and redacts the file sequentially.

4.5. Formal guarantee

Let $S_w(T)$ be the set of detections emitted by Algorithm 1, mapped back to absolute offsets, and $P_{w,k}(T)$ the output of the parallel engine with k workers.

Assumption A1 (Bounded span)**.** *Every detection in* $\Pi(T)$ *has length* $j - i \leq L_{\max}$.

Assumption A2 (Bounded context)**.** *Whether a span* (i, j) *belongs to* $\Pi(T)$*, and its final confidence, depend only on* $T[i - C_{\text{left}} : j + C_{\text{right}}]$.

Theorem 1 (Streaming equivalence)**.** *Under A1 and A2, if*

$$w \geq \max(L_{\max}, C_{\text{left}}, C_{\text{right}}),$$

then $S_w(T) = \Pi(T)$ *for every text* T *and every chunking of* T.

Proof sketch. Completeness: the engine emits only up to w characters before the buffer's end and retreats out of any straddling span, which by A1 is at most $L_{\max} \leq w$ long; a span cut by a chunk boundary is therefore presented whole in a later pass. *Soundness:* each pass detects over the last $w \geq C_{\text{left}}$ emitted characters plus the buffer, and a span is emitted only if it ends at least $w \geq C_{\text{right}}$ characters before the buffer's live end; both contexts are real text, so by A2 every verdict and confidence equals its whole-document value. *Exactly once:* a span is emitted in the unique pass whose boundary first passes its end, and emitted and carried regions are disjoint. □

Theorem 2 (Parallel identity with fallback)**.** *Under A1 with* $w \geq L_{\max}$*,* $P_{w,k}(T)$ *equals the sequential output byte for byte. If A1 is violated at a seam, the seam cross-check detects the disagreement and the engine falls back to sequential processing, so the outputs are equal in every case.*

Theorem 3 (Bounded memory)**.** *The working buffer of* S_w *never exceeds* $m + w + L_{\max}$ *characters, independent of* $|T|$*; the parallel engine uses at most* k *times that.*

Full proofs, with each step mapped to the implementation, are in the repository's formal-model document (`https://github.com/MounishSenisetty/SafeStream-Redactor/blob/0765dae/docs/FORMAL_MODEL.md`). Because redaction is a pure function of T and the detection set, Theorem 1 implies byte-identical redacted output.

Where the constants come from. Each detector declares its maximum span and the left and right context it reads. The engine computes the required overlap as the maximum over active detectors and refuses to start with a smaller overlap unless the caller explicitly overrides it. For the default

pipeline, $C_{\text{left}} = 96$ (the label-anchored cue window), $C_{\text{right}} = 8$, and $L_{\max} = 8192$ (bounds on private-key blocks and entropy candidates), so $w = 8192$ satisfies Theorem 1. A regression test fails if any detector's declaration changes these values, and a differential test probes each detector to check that its real context dependency does not exceed its declaration.

Scope of the guarantee. The theorems hold for detectors whose output on a span depends only on a bounded neighbourhood. The pattern tiers satisfy this by construction. The spaCy tier declares bounds of 256 characters, but a statistical model's output can in principle depend on everything in its input window, so for that tier the bounds are declared, not proved. A detector that needs document-wide context, such as a model that conditions on a whole document, falls outside the guarantee.

4.6. Implementation

The pure-Python implementation is the reference. An optional Rust core implements the pattern tiers. At start-up it checks that its compiled pattern sources equal the Python ones, and it hands a window back to Python in the rare cases where Unicode case folding or floating-point summation could make the two disagree. In our experiments the two backends produced identical predictions on all three test splits (Section 6).

Use of AI assistance. An AI assistant (Claude, Anthropic) was used to implement evaluation scripts and to draft this manuscript under the authors' supervision. The authors reviewed every result and every sentence, and are accountable for the final content.

5. Experimental setup

5.1. Corpora and splits

Table 1 lists the corpora. Each was split once into development and test halves with a fixed seed. The split manifests list document identifiers and the SHA-256 of the source data, and the loader refuses a source that no longer matches its manifest. All development, error analysis and tuning reported here used development halves only, with the exception of RedactionBench, discussed below. Test halves were scored after the code was frozen.

- **presidio-research** [12]: the synthetic dataset distributed with Presidio's evaluation toolkit (revision 0cb3650), with person, location, organisation, telephone, card, date and other labels.

Table 1: Test splits. Gold-free documents contain no gold span; any detection on them is a false alarm.

Corpus	Documents	Gold spans	Gold-free docs	Size (bytes)
presidio-research	750	1,394	63	63,678
secrets	313	293	97	46,595
RedactionBench	100	25,592	0	1,378,267

- **secrets**: the labelled examples in the Nosey Parker rule set [26] (revision 2e6e7f3) as positives, with gold spans given by each rule's own capture groups, plus 200 generated non-secrets (UUIDs, commit hashes, SHA-256 digests, base64 image payloads, minified JavaScript) as gold-free documents. The split is by rule, so the test half contains vendor formats absent from the development half.
- **RedactionBench** [30]: 200 documents across eleven domains with character spans labelled *mandatory* or *contextual*. All 200 documents were used for error analysis and rule writing before the split existed, so its test half is not held out; Section 6.3 quantifies the effect.

5.2. Systems

SafeStream was run with the Rust core and default tiers (*pattern*), with the opt-in date tier (*+dates*), and with the spaCy `en_core_web_sm` tier (*+spaCy*). **Presidio** 2.2.355 [4] was run as its pattern recognisers alone (*regex*) and as the full analyser with spaCy `en_core_web_sm` (*sm*) and `en_core_web_lg` (*lg*), all with spaCy 3.8.13 and model version 3.8.0. **gitleaks** 8.28.0 [5] and **TruffleHog** 3.90.8 [6] were run with default rules; TruffleHog ran with `--no-verification` so that no example credential was sent to a live service. Both scanners were run once over a directory containing one file per document, and their findings were mapped back to character offsets; TruffleHog's composite findings (for example `user:token`) were credited for each component that appears in the text. Each Python tool ran in its own virtual environment and process, pinned to one thread; the two scanners are Go binaries and ran with their default concurrency.

5.3. Metrics

Let G and R be the sets of gold and redacted character positions in a document. Character precision and recall are $|G \cap R|/|R|$ and $|G \cap R|/|G|$,

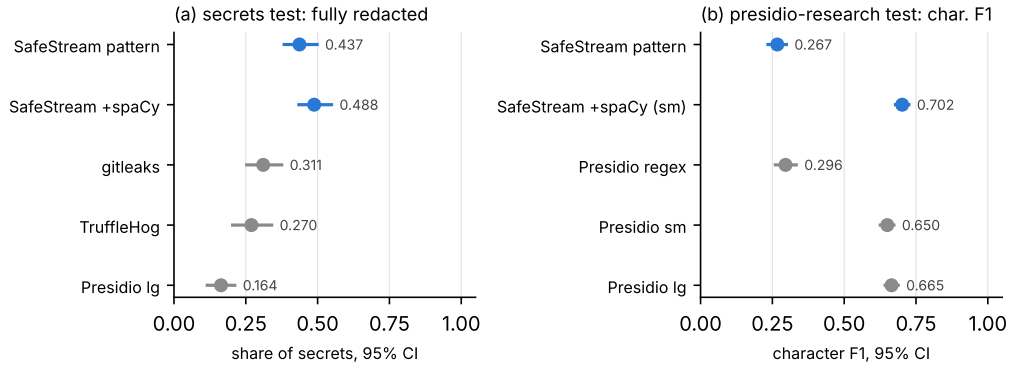


Figure 3: Test-split accuracy with 95% bootstrap intervals. (a) Share of secrets fully redacted on the secrets test split. (b) Character F1 on the presidio-research test split. SafeStream configurations in blue, baselines in grey.

summed over documents, and F1 is their harmonic mean. Because a partly visible value still leaks, the *fully redacted rate* counts a gold span only if every one of its characters is redacted. The *false-alarm document rate* is the share of gold-free documents with at least one detection. On RedactionBench, *mandatory recall* is character recall over mandatory spans, and the *no-leak document rate* is the share of documents whose mandatory characters are all redacted. All metrics are type-agnostic: a value counts as protected whatever label it receives. We report 95% percentile bootstrap intervals over documents (2,000 resamples), and paired differences between systems use the same resamples for both.

5.4. Environment

Accuracy runs used a 4-vCPU Intel Xeon virtual machine at 2.80 GHz with 15 GB RAM, Linux, CPython 3.13.16. Throughput figures (Section 6.7) come from separate runs, five per configuration, on the same machine with no other benchmark running; the throughput reported by the accuracy runs is not used.

6. Results

6.1. Held-out accuracy

Table 2 gives the test-split results and Figure 3 the headline comparisons.

Table 2: Test-split results with 95% bootstrap intervals. SafeStream rows are after the entropy fix of Section 6.2, which changed only the secrets results. FR: fully redacted rate; FA: false-alarm document rate.

Corpus / system	Char P	Char R	Char F1	FR	FA
presidio-research (750 documents)					
SafeStream pattern	.995 [.992,.998]	.154 [.133,.176]	.267 [.235,.299]	.115 [.098,.134]	.000
SafeStream +dates	.996 [.993,.998]	.169 [.146,.192]	.290 [.255,.322]	.129 [.110,.150]	.000
SafeStream +spaCy (sm)	.898 [.877,.918]	.576 [.547,.606]	.702 [.679,.725]	.538 [.509,.568]	.079
Presidio regex	.948 [.919,.974]	.175 [.151,.202]	.296 [.260,.333]	.123 [.105,.145]	.000
Presidio sm	.871 [.848,.893]	.518 [.491,.546]	.650 [.626,.673]	.506 [.479,.536]	.048
Presidio lg	.878 [.856,.900]	.535 [.507,.563]	.665 [.642,.688]	.562 [.534,.590]	.000
secrets (313 documents)					
SafeStream pattern	.857 [.705,.946]	.829 [.740,.887]	.842 [.736,.906]	.437 [.384,.498]	.010
SafeStream +spaCy (sm)	.757 [.604,.858]	.845 [.762,.897]	.799 [.683,.870]	.488 [.435,.548]	.340
gitleaks	.837 [.630,.962]	.473 [.299,.679]	.604 [.430,.765]	.311 [.253,.375]	.000
TruffleHog	.803 [.719,.878]	.189 [.117,.308]	.306 [.204,.446]	.270 [.204,.340]	.000
Presidio regex	.282 [.211,.356]	.105 [.063,.177]	.153 [.098,.225]	.126 [.083,.168]	.392
Presidio sm	.251 [.191,.315]	.119 [.072,.197]	.161 [.107,.232]	.160 [.112,.206]	.649
Presidio lg	.308 [.238,.381]	.131 [.082,.214]	.184 [.124,.262]	.164 [.116,.212]	.433

Personal data. On presidio-research, SafeStream's pattern tiers were precise (character precision 0.995) but recalled only 0.154 of sensitive characters, because most gold spans are names, locations and organisations that patterns cannot capture. Against Presidio's pattern recognisers, the paired difference in character recall was −0.021 [−0.041, −0.002] and in precision +0.047 [+0.021, +0.076]. With the same spaCy model, SafeStream +spaCy exceeded Presidio sm in character F1 by +0.051 [+0.027, +0.076] and in fully redacted rate by +0.031 [+0.004, +0.060]. Against Presidio lg, which uses a larger model, SafeStream +spaCy had higher character F1 (+0.036 [+0.011, +0.064]), but the difference in fully redacted rate, −0.025 [−0.053, +0.004], was not significant.

Credentials. On secrets, SafeStream's pattern tiers fully redacted 0.437 of test secrets, against 0.311 for gitleaks and 0.270 for TruffleHog. The paired differences in fully redacted rate were +0.128 [+0.073, +0.183] and +0.167 [+0.079, +0.250], and in character F1 +0.233 [+0.043, +0.441] and +0.536 [+0.310, +0.685]. The differences in character precision were not significant (+0.005 [−0.063, +0.123] and +0.060 [−0.109, +0.176]). SafeStream flagged one of 97 generated non-secrets; neither scanner flagged any. Presidio has no credential recognisers. Adding spaCy raised SafeStream's false-alarm rate to 0.340; on the development split, the model's false alarms were the word

`Merge` in commit lines tagged as a person and digest or UUID tokens tagged as persons or organisations. The development-split recall (0.542) exceeded the test recall (0.437) because the split is by rule and the test half holds vendor formats for which SafeStream has no pattern.

6.2. False alarms on high-entropy non-secrets

Before the fix described in Section 4, SafeStream flagged 83 of the 103 generated non-secrets in the development split (false-alarm rate 0.806), every one through the entropy tier: all UUIDs, commit hashes, SHA-256 digests and base64 image payloads, and none of the minified JavaScript. The suppression rule was designed on the development split. A first version cost one development secret, a UUID-shaped access key assigned to `SAUCE_ACCESS_KEY`, which led to the secret-key veto. With the veto, the development false-alarm rate fell to 0.000 with no change in recall, and the presidio-research and RedactionBench development results were unchanged except in the fourth decimal. On the test split, the false-alarm rate fell from 0.794 [0.714, 0.872] to 0.010 [0.000, 0.035], character precision rose from 0.589 to 0.857, and recall was unchanged (0.829 character recall, 0.437 fully redacted). The test half had been scored once with the pre-fix code before the fix was made, and both scores are reported. Because the gold-free documents in both halves come from the same generator, the test improvement is an optimistic estimate of the fix's effect on real non-secrets.

6.3. RedactionBench and contamination

Table 3 separates the contaminated and uncontaminated results. Rules written after inspecting the benchmark raised test mandatory recall of the pattern tiers by +0.091 [+0.044, +0.163] and character F1 by +0.060 [+0.029, +0.108] relative to the pre-exposure code. Because those rules were written while the test documents were visible, this gain is an upper bound on what they contribute to unseen data. One of the rules, the label-anchored identifier tier, also improved presidio-research, a corpus it was never tuned on (Table 4), which is evidence that it generalises. Development-set selection chose the same configuration for both code versions (spaCy on, entropy on, confidence threshold 0.3). The pre-exposure code, which never saw the benchmark, still exceeded every Presidio configuration with non-overlapping intervals on two measures: mandatory recall with spaCy (0.674 [0.566, 0.779] against at most 0.420 [0.333, 0.499]) and character F1 of the pattern tier (0.518 [0.434, 0.593]

Table 3: RedactionBench test half (100 documents). *Exposed*: current code, whose rules were written after inspecting all 200 documents. *Pre-exposure*: the last commit before the benchmark entered the repository. *Dev-selected*: configuration chosen on the development half by a fixed rule (maximum development character F1) and scored on the test half once.

System	Char F1	Mandatory recall	No-leak docs
Pre-exposure, pattern	.518 [.434,.593]	.589 [.476,.696]	.074 [.022,.129]
Pre-exposure, +spaCy	.472 [.395,.547]	.674 [.566,.779]	.137 [.073,.208]
Pre-exposure, dev-selected	.469 [.394,.543]	.676 [.568,.781]	.137 [.073,.208]
Exposed, pattern	.579 [.490,.662]	.682 [.578,.787]	.084 [.032,.144]
Exposed, +spaCy	.512 [.429,.590]	.761 [.657,.860]	.179 [.105,.263]
Exposed, dev-selected	.508 [.427,.588]	.762 [.658,.862]	.179 [.105,.263]
gitleaks	.214 [.130,.314]	.247 [.140,.372]	.021 [.000,.053]
TruffleHog	.030 [.017,.048]	.024 [.013,.040]	.000
Presidio regex	.323 [.250,.393]	.331 [.227,.422]	.011 [.000,.033]
Presidio sm	.336 [.265,.402]	.416 [.326,.500]	.084 [.032,.146]
Presidio lg	.344 [.277,.409]	.420 [.333,.499]	.105 [.052,.174]

against at most 0.344 [0.277, 0.409]). No SafeStream configuration redacted every mandatory character in more than 18% of documents.

6.4. Ablation

Table 4 isolates the components. The entropy tier carried most of the credential recall: without it, character F1 on secrets fell from 0.842 to 0.383. The label-anchored tier mattered on presidio-research, where several identifiers carry labels. The contextual scorer's own detections did not change any result on these corpora; its confidence adjustment was kept in every variant. Alone, the deterministic tier reached a character F1 of 0.380 on secrets and 0.237 on presidio-research, and the entropy tier alone 0.822 on secrets. The union rule raised recall by small but mostly significant amounts, at no significant cost in F1 (secrets +0.000 [−0.003, +0.005], RedactionBench +0.003 [−0.002, +0.008]); on presidio-research its F1 difference was +0.004 [+0.001, +0.007]. The effect of the triage gate in front of the ONNX tier was not measured (Section 7).

6.5. Equivalence and backend parity

On every test split, the Rust core and the pure-Python reference produced identical predictions (presidio-research 178 spans, secrets 298, Redac-

Table 4: Ablation on the test splits (pure-Python backend, identical output to the Rust core). Paired differences against the full pattern pipeline, with 95% intervals; FR: fully redacted rate.

Change	secrets	presidio-research	RedactionBench
Character F1, full pattern pipeline minus variant			
Remove entropy tier	+.455 [.242,.641]	.000	+.156 [.104,.224]
Remove label-anchored tier	.000	+.030 [.019,.044]	+.004 [.002,.006]
Remove contextual detections	.000	.000	.000
Union rule versus greedy resolution (with spaCy), union minus greedy			
Character recall	+.006 [.000,.017]	+.005 [.002,.008]	+.009 [.004,.016]
FR / mandatory recall[a]	+.013 [.000,.032]	+.001 [.000,.002]	+.012 [.004,.022]

[a] Fully redacted rate for secrets and presidio-research; mandatory recall for RedactionBench.

tionBench 3,167, from the pre-fix frozen commit). The repository's equivalence suite, which asserts byte-identical streamed, whole-document and parallel output over exhaustive small chunkings, property-based generated inputs, seam-straddling entities and forced fallbacks, plus the native-parity and declared-context tests, passed 133 of 133 tests at the final commit. These tests corroborate the theorems; they do not replace the proofs.

6.6. Memory

Figure 4 plots peak resident memory against input size. Each point is a fresh process redacting a generated log file dense with personal data and credentials, and reports the process's peak resident set size minus its resident set size after imports, so allocations in the Rust core are included. With the Rust core, streaming `redact_file` added between 8.4 and 8.54 MB at every size from 8 MiB to 2 GiB, a 256-fold range. The pure-Python backend added between 4.3 and 4.4 MB from 8 to 512 MiB; it was not run at larger sizes because of its run time (726 s at 512 MiB). Whole-document `redact()` on the same inputs added 235 MB at 8 MiB and 905 MB at 32 MiB, about 28 times the input size, because the input string, the detections and the output are all held at once. A 128 MiB whole-document run was stopped after 20 minutes without completing and is not reported. These measurements are consistent with Theorem 3; they cover one generated input shape on one machine.

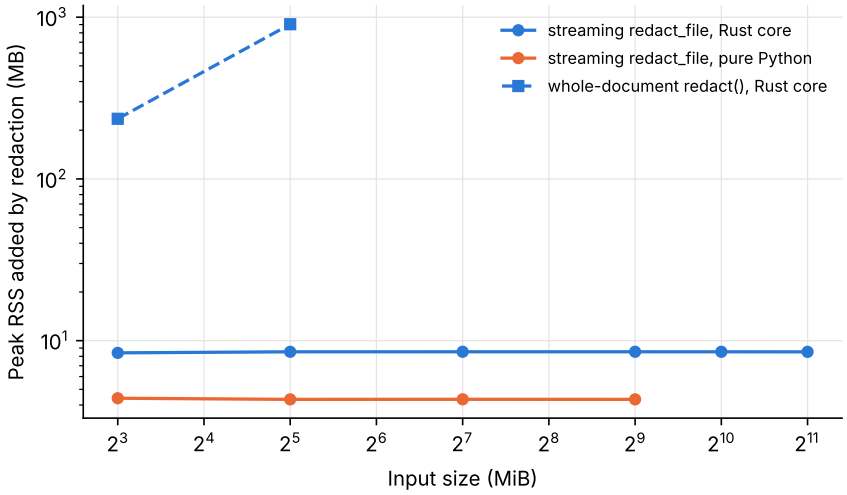


Figure 4: Peak resident memory added by redaction (peak RSS minus RSS after imports) against input size, log scales, each point a fresh process. Streaming `redact_file` stays flat from 8 MiB to 2 GiB (Rust core) and to 512 MiB (pure Python); whole-document `redact()` grows with the input.

6.7. Throughput

Table 5 reports throughput on the test splits. Per-document overhead dominates on presidio-research and secrets, whose documents average 85 and 149 bytes; RedactionBench's documents are larger (13.8 kB on average) and are closer to the files SafeStream targets. There, the Rust core processed 10.4 ± 1.3 MB/s, the pure-Python reference 1.23 ± 0.15 MB/s and Presidio's pattern recognisers 0.069 MB/s. With a statistical model in the loop, every system ran below 0.05 MB/s: SafeStream +spaCy at 0.047 MB/s and Presidio sm and lg at 0.020 and 0.020 MB/s.

On a single 256 MB generated log file (mostly benign log lines and prose with personal data and credentials mixed in), `redact_file` with the Rust core ran at 10.5 ± 0.5, 20.6 ± 0.5 and 41.1 ± 1.4 MB/s with 1, 2 and 4 worker processes, a 3.9-fold speed-up on 4 vCPUs (Figure 5). The pure-Python backend on a 32 MB file of the same kind ran at 0.99, 1.90 and 2.67 MB/s. These figures come from one virtual machine and should be read as relative rather than absolute.

Table 5: Throughput on the test splits in MB/s, mean ± sample standard deviation over 5 runs, one thread, load time excluded. Each run is a fresh process on an otherwise idle 4-vCPU machine.

System	presidio-research	secrets	RedactionBench
SafeStream pattern (Rust core)	1.82 ± 0.42	2.39 ± 0.58	10.4 ± 1.3
SafeStream pattern (pure Python)	1.1 ± 0.07	0.916 ± 0.17	1.23 ± 0.15
SafeStream +spaCy (sm)	0.0118 ± 0.0009	0.0227 ± 0.0034	0.0467 ± 0.0033
Presidio regex	0.0802 ± 0.0127	0.0983 ± 0.0155	0.0694 ± 0.0028
Presidio sm	0.0081 ± 0.0004	0.0125 ± 0.0007	0.0201 ± 0.0003
Presidio lg	0.0074 ± 0.0010	0.0128 ± 0.0008	0.0203 ± 0.0003
gitleaks[a]	0.507 ± 0.028	0.409 ± 0.051	3.15 ± 0.19
TruffleHog[a]	0.0269 ± 0.0016	0.0188 ± 0.0007	0.594 ± 0.02

[a] Go binaries run once per corpus over a directory of documents; the figure includes process start-up and is not limited to one thread.

7. Discussion and limitations

What the results support. The equivalence and memory properties are proven for the pattern tiers and corroborated by tests and by the memory curve. The pattern tiers are fast enough for large files, and the parallel engine scaled almost linearly to the four available cores; adding a statistical model reduced throughput by two orders of magnitude, for SafeStream and Presidio alike. On credentials, the pattern tiers covered more secrets than two widely used scanners at similar precision. On personal data, SafeStream with a spaCy model was more accurate than Presidio with the same model on one synthetic corpus. Without a named-entity model, its free-text recall is low.

Practical implications. For operators, the results suggest a two-stage deployment. The pattern tiers can run on every byte of a log or export, at 10.5 MB/s per worker and 41.1 MB/s on four workers in our measurements, with bounded memory and a guarantee that chunking introduces no leak, and they cover credentials better than the two scanners we tested. A named-entity tier closes much of the free-text gap but costs two orders of magnitude in throughput, so it is best applied where names are expected (tickets, e-mail, free-text fields) rather than to whole machine logs. Because the guarantee is tied to declared context bounds, adding a custom detector is safe exactly when its declaration is truthful; the engine refuses overlaps smaller than the declarations require.

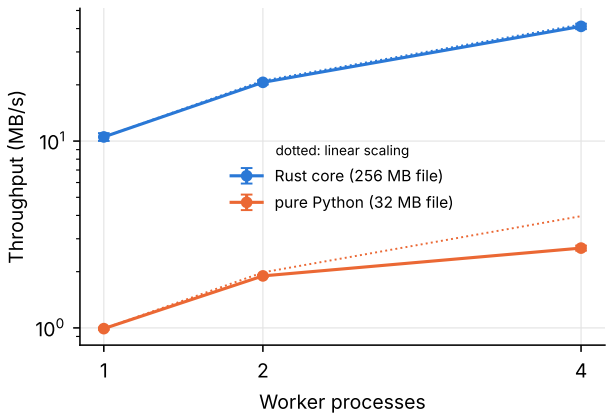


Figure 5: Throughput of `redact_file` against worker processes on a 4-vCPU machine, mean and standard deviation over 5 runs (log scale). Dotted lines show linear scaling from one worker. The parallel output is byte-identical to the sequential output (Theorem 2).

What was not evaluated. The ONNX DistilBERT tier, its triage gate, Presidio's transformer configurations, the ai4privacy corpus [31] and CoNLL-2003 were not run, because their models and data are hosted on a service that the evaluation environment could not reach. We make no claim about them. No large-language-model baseline was evaluated.

Threats to validity. presidio-research is synthetic. The secrets gold spans are Nosey Parker's capture groups, so credentials adjacent to a labelled secret (for example a client identifier next to its secret) count as false positives, and the gold-free documents are generated. RedactionBench's test half is contaminated (Section 6.3), and its contextual labels have low human agreement according to its authors [30]. All runs used one machine. The evaluation is English-only.

Limits of the guarantee. The theorems require that every detector declare a true upper bound on the context it reads. A detector that understates its dependency, or one that needs document-wide context, is outside the guarantee. Lowering the overlap below the declared requirement is refused by default; if a caller overrides that, the parallel engine still falls back safely, but the sequential engine can split a long span.

8. Conclusion

SafeStream-Redactor shows that chunked redaction of personal data and credentials need not trade correctness for scale: with detector-declared bounds on span length and context, a bounded overlap makes the streamed result provably equal to the whole-document result, a seam cross-check makes the parallel result equal to the sequential one, and working memory does not grow with input size. On held-out test splits, its pattern tiers covered more credentials than gitleaks and TruffleHog, and with a spaCy model it was more accurate than Presidio with the same model on a synthetic personal-data corpus. Future work is to evaluate the neural tier and its triage gate, to extend the guarantee's empirical checks to statistical detectors, and to test on real, non-synthetic corpora with credentials and personal data together.

CRediT authorship contribution statement

[MISSING: each author's CRediT roles (Conceptualization, Software, Validation, Formal analysis, Investigation, Writing – original draft, Writing – review & editing, Supervision)]

Declaration of competing interest

[MISSING: The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. – confirm or replace]

Funding

[MISSING: funding sources, or: This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.]

Data availability

Source code, split manifests, evaluation harness and raw results are available at `https://github.com/MounishSenisetty/SafeStream-Redactor` (branch `phase2-evaluation-and-paper`; every number in this article is generated from files under `research/results/`). [MISSING: archive a tagged

release (e.g. on Zenodo) and give its DOI here] presidio-research and Nosey Parker are public; RedactionBench is available from its authors under CC BY 4.0. Generated non-secrets are produced by the released code with a fixed seed.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Claude (Anthropic) to implement evaluation scripts and to draft the manuscript text. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Acknowledgements

[MISSING: acknowledgements, or remove this section]

References

[1] S. Alneyadi, E. Sithirasenan, V. Muthukkumarasamy, A survey on data leakage prevention systems, Journal of Network and Computer Applications 62 (2016) 137–152.

[2] M. Meli, M. R. McNiece, B. Reaves, How bad can it git? characterizing secret leakage in public GitHub repositories, in: Network and Distributed System Security Symposium (NDSS), 2019.
URL `https://www.ndss-symposium.org/ndss-paper/how-bad-can-it-git-characterizing-secret-leakage-in-public-github-repositories/`

[3] A. Krause, J. H. Klemmer, N. Huaman, D. Wermke, Y. Acar, S. Fahl, Pushed by accident: A mixed-methods study on strategies of handling secret information in source code repositories, in: 32nd USENIX Security Symposium, 2023, pp. 2527–2544.

[4] Microsoft, Presidio: Data protection and de-identification SDK, `https://github.com/microsoft/presidio`, software; version 2.2.355 evaluated. Accessed 2026-10-11 (2026).

[5] Gitleaks contributors, Gitleaks, https://github.com/gitleaks/gitleaks, software, version 8.28.0 evaluated. Accessed 2026-10-11 (2026).

[6] Truffle Security, TruffleHog, https://github.com/trufflesecurity/trufflehog, software, version 3.90.8 evaluated with --no-verification. Accessed 2026-10-11 (2026).

[7] S. K. Basak, J. Cox, B. Reaves, L. Williams, A comparative study of software secrets reporting by secret detection tools, in: ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), 2023. arXiv:2307.00714.

[8] I. Neamatullah, M. M. Douglass, L.-w. H. Lehman, A. Reisner, M. Villarroel, W. J. Long, P. Szolovits, G. B. Moody, R. G. Mark, G. D. Clifford, Automated de-identification of free-text medical records, BMC Medical Informatics and Decision Making 8 (2008) 32. doi:10.1186/1472-6947-8-32.

[9] B. Norgeot, K. Muenzen, et al., Protected health information filter (Philter): accurately and securely de-identifying free-text clinical notes, npj Digital Medicine 3 (2020) 57. doi:10.1038/s41746-020-0258-y.

[10] A. Stubbs, Ö. Uzuner, Annotating longitudinal clinical narratives for de-identification: The 2014 i2b2/UTHealth corpus, Journal of Biomedical Informatics 58 (Suppl) (2015) S20–S29. doi:10.1016/j.jbi.2015.07.020.

[11] F. Dernoncourt, J. Y. Lee, Ö. Uzuner, P. Szolovits, De-identification of patient notes with recurrent neural networks, Journal of the American Medical Informatics Association 24 (3) (2017) 596–606. doi:10.1093/jamia/ocw156.

[12] Microsoft, presidio-research: Toolbox for developing and evaluating PII detectors, https://github.com/microsoft/presidio-research, synthetic dataset synth_dataset_v2.json, revision 0cb3650. Accessed 2026-10-11 (2026).

[13] P. Lison, I. Pilán, D. Sánchez, M. Batet, L. Øvrelid, Anonymisation models for text data: State of the art, challenges and future directions, in: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on

Natural Language Processing (Volume 1: Long Papers), 2021, pp. 4188–4203. doi:10.18653/v1/2021.acl-long.323.

[14] I. Pilán, P. Lison, L. Øvrelid, A. Papadopoulou, D. Sánchez, M. Batet, The text anonymization benchmark (TAB): A dedicated corpus and evaluation framework for text anonymization, Computational Linguistics 48 (4) (2022) 1053–1101. doi:10.1162/coli_a_00458.

[15] E. F. Tjong Kim Sang, F. De Meulder, Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition, in: Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003, 2003, pp. 142–147.
URL https://aclanthology.org/W03-0419

[16] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), 2019, pp. 4171–4186. doi:10.18653/v1/N19-1423.

[17] V. Sanh, L. Debut, J. Chaumond, T. Wolf, DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter (2019). arXiv:1910.01108.

[18] U. Zaratiana, N. Tomeh, P. Holat, T. Charnois, GLiNER: Generalist model for named entity recognition using bidirectional transformer, in: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), 2024, pp. 5364–5376. doi:10.18653/v1/2024.naacl-long.300.

[19] H. Krawczyk, M. Bellare, R. Canetti, HMAC: Keyed-hashing for message authentication, RFC 2104 (1997). doi:10.17487/RFC2104.

[20] L. Sweeney, k-anonymity: A model for protecting privacy, International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems 10 (5) (2002) 557–570.

[21] V. S. Sinha, D. Saha, P. Dhoolia, R. Padhye, S. Mani, Detecting and mitigating secret-key leaks in source code repositories, in: Proceedings of

the 12th Working Conference on Mining Software Repositories (MSR), 2015.

[22] A. Saha, T. Denning, V. Srikumar, S. K. Kasera, Secrets in source code: Reducing false positives using machine learning, in: 2020 International Conference on COMmunication Systems & NETworkS (COMSNETS), 2020. doi:10.1109/COMSNETS48256.2020.9027350.

[23] R. Feng, Z. Yan, S. Peng, Y. Zhang, Automated detection of password leakage from public GitHub repositories, in: Proceedings of the 44th International Conference on Software Engineering (ICSE), 2022.

[24] S. K. Basak, L. Neil, B. Reaves, L. Williams, SecretBench: A dataset of software secrets, in: Proceedings of the 20th International Conference on Mining Software Repositories (MSR), Data and Tool Showcase Track, 2023. arXiv:2303.06729.

[25] C. E. Shannon, A mathematical theory of communication, Bell System Technical Journal 27 (3) (1948) 379–423. doi:10.1002/j.1538-7305.1948.tb01338.x.

[26] Praetorian, Nosey parker, https://github.com/praetorian-inc/noseyparker, rule set at revision 2e6e7f3, Apache-2.0. Accessed 2026-10-11 (2026).

[27] S. Muthukrishnan, Data streams: Algorithms and applications, Foundations and Trends in Theoretical Computer Science 1 (2) (2005) 117–236. doi:10.1561/0400000002.

[28] K. Thompson, Regular expression search algorithm, Communications of the ACM 11 (6) (1968) 419–422. doi:10.1145/363347.363387.

[29] H. P. Luhn, Computer for verifying numbers, US Patent 2,950,048 (1960).

[30] S. Brynjólfsson, et al., RedactionBench (2026). arXiv:2606.18782.

[31] AI4Privacy, pii-masking-300k, https://huggingface.co/datasets/ai4privacy/pii-masking-300k, dataset, revision c8c7789; academic non-commercial licence (2026).